

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Лабораторная работа №1
по курсу “Компьютерная графика”**

Основы 3D графики

Выполнил: Д. А. Широбоков
Группа: М8О-301БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель лабораторной работы: познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, научиться работать с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

Требования: Программа должна работать в реальном времени, с возможностью динамической смены проекции и трансформаций объектов. Все объекты должны корректно отрисовываться с учетом проекции и иметь возможность взаимодействия с пользователем.

Вариант №6 — Пирамида.

Дополнительные задания:

1. Обеспечьте переключение матрицы проекции (ортографическая и перспективная) в интерфейсе
2. Добавить элементы интерфейса для изменения позиции, поворота и растяжения фигуры
3. Добавить возможность движения и поворота фигуры по сложной траектории, сделать элемент интерфейса для паузы и воспроизведения “анимации”, а также возможность изменять скорость “анимации” и другие параметры траектории (например радиус траектории и др.)
4. Добавить возможность изменять цвет фигуры с помощью элементов интерфейса (в ImGui есть ColorEdit для этого)
5. Назначить процедурно цвета для каждой вершины, например в зависимости от позиции вершин в локальной системе координат фигуры, тогда возможность изменять цвет с помощью элементов интерфейса будет умножать выбранный цвет на цвет вершин

Метод решения

1. Геометрия объекта

Пирамида задана 5 вершинами в локальной системе координат (центр основания в начале, высота 1, основание — квадрат со стороной 1):

Индекс	Позиция (x, y, z)	Назначение
0	(0.5, -0.5, 0.5)	Основание, правый передний
1	(0.5, -0.5, -0.5)	Основание, правый задний
2	(-0.5, -0.5, -0.5)	Основание, левый задний
3	(-0.5, -0.5, 0.5)	Основание, левый передний
4	(0.0, 0.5, 0.0)	Вершина

Индексный буфер (18 индексов, 6 треугольников) описывает 4 боковые грани и 2 треугольника основания. Порядок вершин выбран так, чтобы при VK_FRONT_FACE_CLOCKWISE и отражении матрицы вида по Y лицевые грани имели правильную ориентацию.

Цвет вычисляется на CPU из локальной позиции вершины:

```
vec3 proceduralVertexColor(vec3 p) {
    float h = p.y + 0.5f;           // 0..1 внутри фигуры
    return vec3(1.0 - p.x,          // R: градиент по X (красный → синий)
                0.5 + 0.5 * h,      // G: яркость по высоте
                p.x + 0.5           // B: градиент по X (синий → красный)
    )
}
```

Во фрагментном шейдере этот цвет модулируется пользовательским tint (ImGui ColorEdit3):

```
outColor = vec4(fragColor * pc.color, 1.0);
```

2. Аффинные преобразования модели

Модельная матрица собирается в порядке (справа налево):

$$M = T(\text{objectPosition}) \cdot R_{\text{orientation}} \cdot S(\text{scale}) \cdot R_x \cdot R_y \cdot R_z$$

- scale, rotationDegrees, position — управляются ползунками ImGui.
- objectPosition = state.position + orbitOffset(t) — базовая позиция + смещение по траектории.
- R_orientation — ортонормированная матрица, ориентирующая пирамиду вдоль траектории:
 - Локальная ось Y (вершина) совпадает с касательной к спирали tangent(t).
 - Локальная диагональ (X+Z)/√2 смотрит к центру спирали (inward).
 - Ось X = (inward + side)/√2, ось Z = (inward - side)/√2, где side = normalize(cross(tangent, inward)).

3. Траектория движения (спираль)

Параметризация прогрессом $p \in [0, 1]$:

$$\begin{aligned}x(p) &= R \cdot \cos(2\pi \cdot \text{turns} \cdot p) \\y(p) &= H \cdot \sin(\pi \cdot p) \quad // H = \text{spiralHalfHeight} = 5 \\z(p) &= R \cdot \sin(2\pi \cdot \text{turns} \cdot p)\end{aligned}$$

Касательная (производная по p , не нормализованная):

$$\begin{aligned}x'(p) &= -R \cdot 2\pi \cdot \text{turns} \cdot \sin(2\pi \cdot \text{turns} \cdot p) \\y'(p) &= H \cdot \pi \cdot \cos(\pi \cdot p) \\z'(p) &= R \cdot 2\pi \cdot \text{turns} \cdot \cos(2\pi \cdot \text{turns} \cdot p)\end{aligned}$$

Длина пути за цикл: $L = 2\pi \cdot R \cdot \text{turns}$.

Физическая скорость $v = \text{speed_slider}$ (ед/сек). Длительность цикла:

$$T = L / v = 2\pi \cdot R \cdot \text{turns} / \text{speed}$$

Таким образом изменение R или turns меняет форму траектории и период, но **не** скорость движения.

Анимационное время `animationTime` накапливается каждый кадр (`deltaTime`), по модулю T . Ползунок «Таймлайн» позволяет вручную задавать прогресс; при перетаскивании анимация ставится на паузу (`timelineDragging`).

4. Камера и проекции

Камера статична: глаз в $(0, 0, \text{distance})$, цель в $(0, 0, 0)$, верх $(0, 1, 0)$.

Матрица вида: $V = \text{lookAt}(\text{eye}, \text{target}, \text{up})$ с последующим отражением $V[1][1] *= -1$.

4.1. Перспективная проекция

$$P_{\text{persp}} = \text{perspective}(\text{fovY}=45^\circ, \text{aspect}=W/H, \text{near}=0.1, \text{far}=\text{farPlane})$$

`farPlane` и `distance` подбираются так, чтобы вся спираль (цилиндр радиуса `maxOrbitRadius=5`, высоты $2 \cdot \text{spiralHalfHeight}=10$ + запас на габарит пирамиды) гарантированно попадала в фрустум:

$$\begin{aligned}\tan V &= \tan(\text{fovY}/2), \quad \tan H = \tan V \cdot \text{aspect} \\dV &= \text{maxOrbitRadius} + (H + \text{allowance}) / \tan V \\dH &= \text{maxOrbitRadius} + (R + \text{allowance}) / \tan H \\distance &= \text{framePadding} \cdot \max(dV, dH) \quad // \text{framePadding} = 1.05 \\farPlane &= distance + H + 1\end{aligned}$$

4.2. Ортогографическая проекция

$P_{ortho} = \text{ortho}(-W, W, -H, H, \text{near}=0.1, \text{far}=\text{farPlane})$

$W = \text{maxOrbitRadius} + \text{allowance}$

$H = \text{spiralHalfHeight} + \text{allowance}$

Размер видимого объёма не зависит от distance, поэтому кадр стабилен при переключении режимов.

Переключение state.perspective (чекбокс ImGui) меняет используемую матрицу проекции в реальном времени.

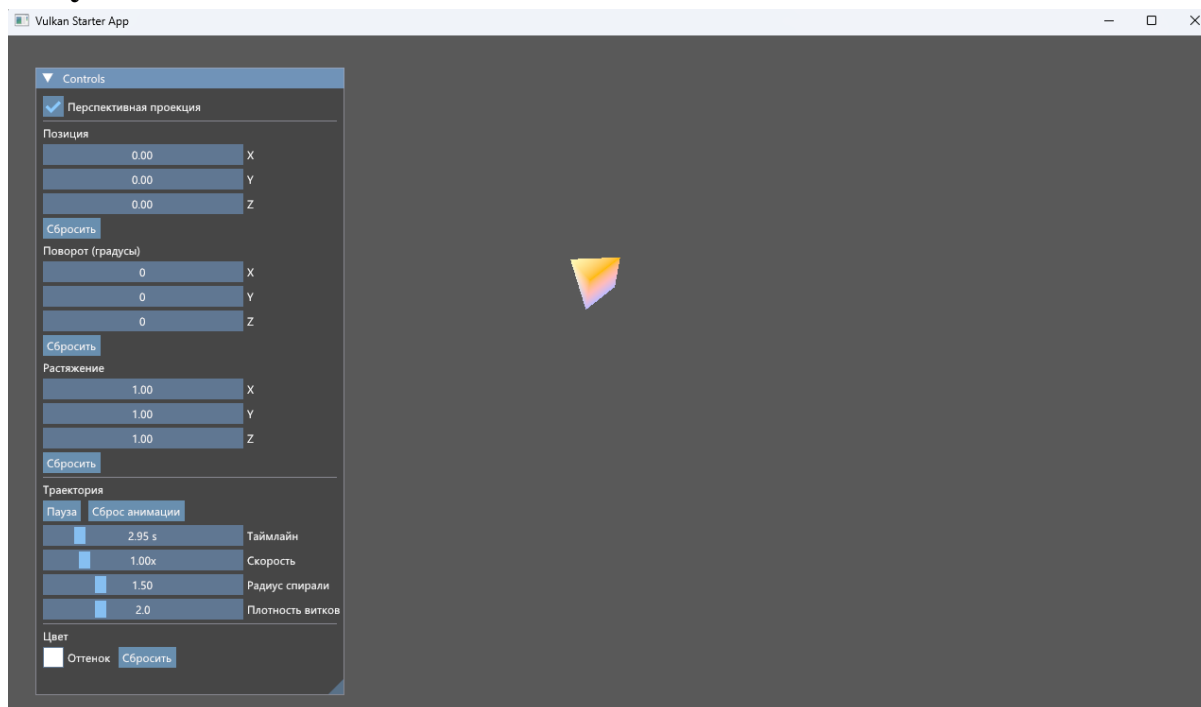
5. Vulkan-объекты и конвейер

Объект	Назначение
VkInstance, VkPhysicalDevice, VkDevice	Инициализация через vk-bootstrap (validation layers, discrete GPU preference).
VkSurfaceKHR (GLFW)	Окно для презентации.
VkSwapchainKHR (FIFO)	Двойная/тройная буферизация, VK_PRESENT_MODE_FIFO_KHR.
VkRenderPass (2 attachment)	Цветной (swapchain image) + глубинный (D24_UNORM_S8_UINT).
VkPipelineLayout + VkPipeline	Графический конвейер: vertex + fragment, VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST, back-face culling (CULL_MODE_BACK_BIT, FRONT_FACE_CLOCKWISE), depth test LESS.
Push Constants (76 байт)	mat4 modelViewProjection (vertex) + vec3 tint (fragment). Обновляются каждый кадр vkCmdPushConstants.
VkBuffer (vertex/index) + VMA (CPU_TO_GPU)	Статические геометрические данные (host-visible, запись при инициализации).
VkCommandPool + VkCommandBuffer	Запись команд рендера и ImGui.
VkSemaphore / VkFence	Синхронизация: image_available → render → image_finished → present → frame_in_flight fence.
ImGui Vulkan backend	Отдельный VkRenderPass (loadOp=LOAD), дескрипторный пул, командный буфер. Отрисовка UI поверх 3D-сцены.

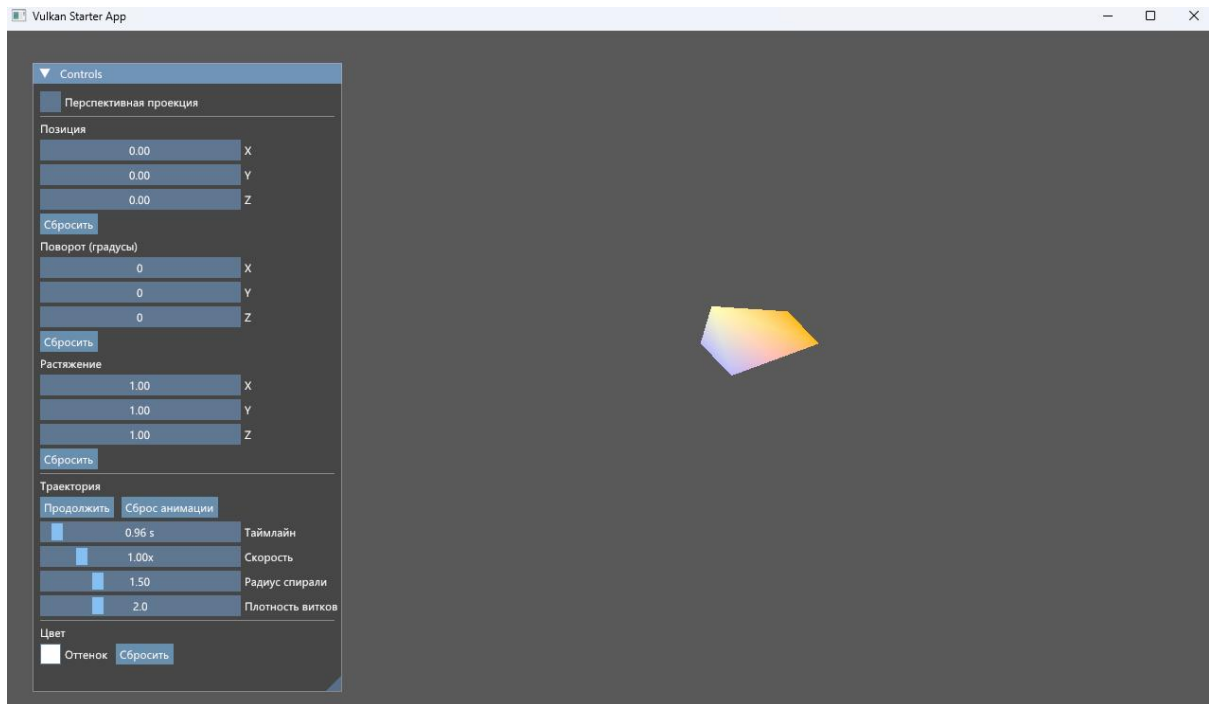
6. Взаимодействие с пользователем

Элемент	Реализация
Вращение мышью	GLFW_MOUSE_BUTTON_LEFT + glfwSetCursorPosCallback; дельта курсора $\rightarrow$ rotationDegrees.x/y (clamp pitch $\pm 89^\circ$). Игнорируется, если ImGui::GetIO().WantCaptureMouse.
ImGui панель	Checkbox, DragFloat, SliderFloat, ColorEdit3, Button. Все поля связаны с SceneState.
Горячие сбросы	Кнопки «Сбросить» возвращают position/rotation/scale/tint к исходным значениям.

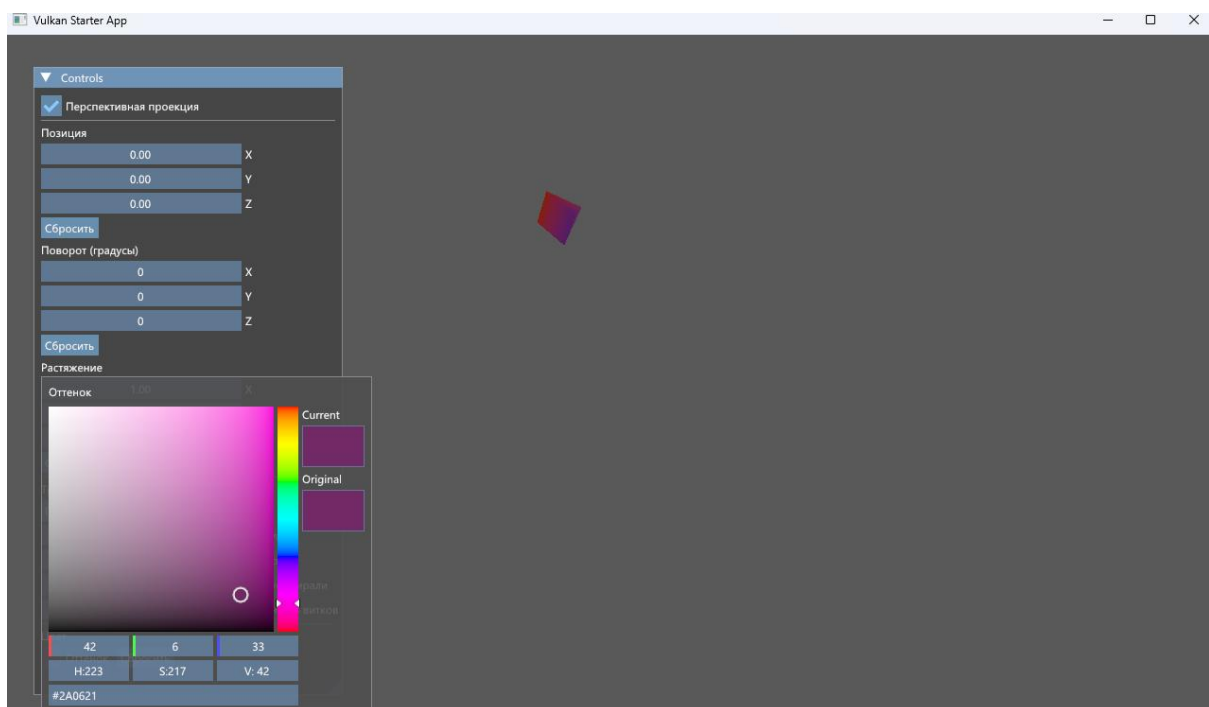
Результаты



Перспективная проекция



Ортогографическая проекция



Смена оттенка

Выводы

В ходе работы были освоены базовые принципы трехмерной графики, а также низкоуровневая инициализация графического конвейера Vulkan (работа со Swapchain, Render Pass, Pipeline и буферами геометрии) с использованием библиотеки vk-bootstrap для ускорения настройки.

1. **Аффинные преобразования и визуализация:** Реализовано корректное построение 3D-объекта (пирамиды) и управление его состоянием в реальном времени. Матрица модели успешно формируется через последовательное масштабирование, вращение и перенос. Процедурное текстурирование на основе локальных координат вершин позволило добиться плавного цветового градиента, эффективно дополненного шейдерным смешиванием.
2. **Сложная кинематика и анимация:** Реализовано параметрическое движение пирамиды по спиральной траектории с физически корректным сохранением линейной скорости при изменении геометрии пути (радиуса и числа витков). Расчет матрицы ориентации обеспечивает естественное выравнивание локальных осей фигуры (вершины и диагонали основания) относительно вектора касательной и центра спирали.
3. **Интерактивный UI и синхронизация:** Интеграция библиотеки ImGui и обработка ввода мыши через GLFW обеспечили полноценное взаимодействие с пользователем (управление трансформациями, цветом, таймлайном и параметрами анимации). Разделение ввода гарантирует, что взаимодействие с интерфейсом не вызывает ложного срабатывания вращения сцены мышью.